GOOGLE APPS SCRIPT · TECHNICAL REFERENCE

Google Calendar *Audit Tool*

Complete installation guide, annotated source code, and a full reference for every function, method, and Apps Script concept used.

Apps Script | Google Sheets | CalendarApp API | JavaScript

CONTENTS

01 Installation Instructions

1 Open Google Sheets

Create a new blank spreadsheet. Navigate to **Extensions > Apps Script** to open the script editor.

2 Paste & Save the Script

Delete any placeholder code in the editor. Paste the full script from Section 02 below. Click the **Save icon** and name the project `CalendarAudit`.

3 Authorize & Run

Refresh your Google Sheet. A new **"My Audit Tools"** menu will appear in the toolbar. Follow the Google OAuth prompts — if prompted about an unverified app, click **Advanced** then **Proceed**.

> ⚠️ **Authorization Note**
>
> Google may display an "unverified app" screen for personal scripts. This is expected. Click **Advanced**, then **Proceed**. The script only reads your own calendar and writes to your own Sheet — no data leaves your Google account.

02 The Annotated Script

Three logical functions — paste all three into a single Apps Script file. Italic lines are comments.

BLOCK 1

Menu Trigger — onOpen()

Runs automatically when the Sheet is opened. Injects a custom "My Audit Tools" menu tab into the Sheets toolbar.

CalendarAudit.gs

```
/**
 * THE TRIGGER
 * This function runs automatically when the Sheet is opened.
 * It injects a custom menu tab into the Google Sheets toolbar.
 */
function onOpen() {
```

```
  var ui = SpreadsheetApp.getUi();
  ui.createMenu('My Audit Tools')
      .addItem('Audit Calendar (Export Events)', 'promptForCalendar')
      .addToUi();
}
```

BLOCK 2

Input Dialogue — promptForCalendar()

Shows a popup asking how many days to scan. Validates and converts the input, then calls the main export function.

CalendarAudit.gs

```
/**
 * THE INPUT DIALOGUE
 * Pops up a dialog asking how many days to scan.
 * Converts text input to an integer for processing.
 */
function promptForCalendar() {
  var ui = SpreadsheetApp.getUi();
  var response = ui.prompt(
    'Calendar Audit',
    'How many days into the future should I look?',
    ui.ButtonSet.OK_CANCEL
  );
```

```
  if (response.getSelectedButton() == ui.Button.OK) {
    var days = parseInt(response.getResponseText()); // Convert text to number
    if (isNaN(days) || days < 1) {
      ui.alert('Please enter a valid number.');
    } else {
      runCalendarAudit(days); // Pass the number to the main logic
    }
  }
}
```

BLOCK 3

Data Exporter — runCalendarAudit(days)

The core engine. Clears the sheet, fetches all events in the date range, calculates durations, and writes everything in a single batch operation for performance.

CalendarAudit.gs

```
/**
 * THE DATA EXPORTER
 * Connects to your default Google Calendar, fetches all events
 * within the specified range, and writes them to the sheet.
 */
function runCalendarAudit(days) {
  var sheet = SpreadsheetApp.getActiveSheet();
  sheet.clear(); // Wipe previous data to start fresh
```

```
  // Create bold column headers
  var headers = [
    'Event Title', 'Start Time', 'End Time',
    'Duration (Hrs)', 'All Day?', 'Location', 'My Status'
  ];
  sheet.appendRow(headers);
  sheet.getRange(1, 1, 1, headers.length)
       .setFontWeight('bold').setBackground('#eeeeee');

  // Calculate the timeframe starting from right now
  var start = new Date();
  var end   = new Date();
  end.setDate(end.getDate() + days);

  // Fetch all events from your default calendar
  var calendar = CalendarApp.getDefaultCalendar();
  var events   = calendar.getEvents(start, end);
  var eventList = [];

  // Loop through every event found
  for (var i = 0; i < events.length; i++) {
    var e = events[i];
    var myStatus = '';

    // Check acceptance status (Owner if getMyStatus() throws)
    try { myStatus = e.getMyStatus(); } catch(err) { myStatus = 'Owner'; }
```

```
      // Convert ms difference to hours (24 for all-day events)
      var duration = e.isAllDayEvent()
        ? '24'
        : ((e.getEndTime() - e.getStartTime()) / 3600000).toFixed(2);

      eventList.push([
        e.getTitle(), e.getStartTime(), e.getEndTime(),
        duration, e.isAllDayEvent(), e.getLocation(), myStatus
      ]);
    }

    // Batch-write all rows at once for performance
    if (eventList.length > 0) {
      sheet.getRange(2, 1, eventList.length, eventList[0].length)
           .setValues(eventList);
      SpreadsheetApp.getUi().alert('Success! Found ' + eventList.length + ' events.');
    } else {
      SpreadsheetApp.getUi().alert('No events found for this timeframe.');
    }
  }
```

03 Output Column Reference

COLUMN	TYPE	DESCRIPTION
`Event Title`	`String`	The display name of the calendar event.
`Start Time`	`DateTime`	When the event begins (your local timezone).
`End Time`	`DateTime`	When the event ends (your local timezone).
`Duration (Hrs)`	`Decimal`	Hours between start and end. All-day events are assigned a value of 24.
`All Day?`	`Boolean`	TRUE if the event spans a full day with no specific time set.
`Location`	`String`	The location field from the calendar event (may be empty).
`My Status`	`String`	Your RSVP status: ACCEPTED, DECLINED, MAYBE, or Owner.

04 Tips & Troubleshooting

💡 Calculating Total Meeting Hours

For recurring meetings, each individual instance is exported as a separate row. Use `=SUM()` on the Duration (Hrs) column to calculate total time spent in meetings across the full export.

💡 Auditing a Different Calendar

To target a specific calendar instead of your default, replace `CalendarApp.getDefaultCalendar()` with `CalendarApp.getCalendarById('your-id@group.calendar.google.com')`. Find the ID under Google Calendar Settings > your calendar > Integrate calendar.

⚠️ Data is Cleared on Every Run

The script calls `sheet.clear()` at the start of each run, permanently erasing all existing content. Copy any previous export to a separate tab before running again if you need to preserve it.

05 Apps Script — Functions, Methods & Concepts

Every Apps Script service, method, and JavaScript concept used in the code above — a reference for when you modify or extend the script.

A SpreadsheetApp Service

The top-level service for interacting with Google Sheets. Provides access to the spreadsheet's UI, data, and structure.

METHOD	SERVICE	RETURNS	WHAT IT DOES
`SpreadsheetApp.getUi()`	`SpreadsheetApp`	`Ui`	Gets the UI environment to create menus, dialogs, and prompts.
`SpreadsheetApp.getActiveSheet()`	`SpreadsheetApp`	`Sheet`	Returns the sheet the user currently has open and visible.
`ui.createMenu(name)`	`Ui`	`Menu`	Creates a new top-level menu with the given label in the toolbar.
`.addItem(label, fn)`	`Menu`	`Menu`	Adds a clickable item that calls the named function when selected.
`.addToUi()`	`Menu`	`void`	Finalises and renders the menu in the spreadsheet toolbar.
`ui.prompt(title, msg, btns)`	`Ui`	`PromptResponse`	Opens a dialog box with a text input field. Returns the user's response object.
`ui.ButtonSet.OK_CANCEL`	`Ui`	`ButtonSet`	A constant that configures the dialog to show OK and Cancel buttons.
`response.getSelectedButton()`	`PromptResponse`	`Button`	Returns which button the user clicked (Button.OK, Button.CANCEL, etc.).

METHOD	SERVICE	RETURNS	WHAT IT DOES
`response.getResponseText()`	`PromptResponse`	`String`	Returns the text the user typed into the dialog's input field.
`ui.alert(message)`	`Ui`	`void`	Displays a simple alert popup with a message and an OK button.
`sheet.clear()`	`Sheet`	`Sheet`	Deletes all content and formatting from every cell on the sheet.
`sheet.appendRow(values[])`	`Sheet`	`Sheet`	Appends a new row at the bottom of the data with the supplied array of values.
`sheet.getRange(row,col,r,c)`	`Sheet`	`Range`	Returns a Range object for the rectangle of cells at (row, col) spanning r rows and c columns.
`range.setFontWeight(w)`	`Range`	`Range`	Sets font weight on the range. Use `'bold'` or `'normal'`.
`range.setBackground(color)`	`Range`	`Range`	Sets the background fill color using a hex string (e.g. `'#eeeeee'`).
`range.setValues(array2D)`	`Range`	`Range`	Writes a 2D array into the range in one batch call — much faster than writing row by row.

B CalendarApp Service

Provides access to the user's Google Calendar — fetch calendars, query events, and read details like title, time, location, and RSVP status.

METHOD	SERVICE	RETURNS	WHAT IT DOES
`CalendarApp.getDefaultCalendar()`	`CalendarApp`	`Calendar`	Returns the user's primary Google Calendar (matching their account email).
`CalendarApp.getCalendarById(id)`	`CalendarApp`	`Calendar`	Returns a specific calendar by its unique ID string. Useful for shared or secondary calendars.
`calendar.getEvents(start, end)`	`Calendar`	`CalEvent[]`	Fetches all events within the start and end Date objects. Returns an array of CalendarEvent objects.
`event.getTitle()`	`CalendarEvent`	`String`	Returns the title (display name) of the event as it appears on the calendar.
`event.getStartTime()`	`CalendarEvent`	`Date`	Returns a JavaScript Date object representing when the event starts.
`event.getEndTime()`	`CalendarEvent`	`Date`	Returns a JavaScript Date object representing when the event ends.

METHOD	SERVICE	RETURNS	WHAT IT DOES
`event.isAllDayEvent()`	`CalendarEvent`	`Boolean`	Returns true if the event is an all-day event with no specific start/end time.
`event.getLocation()`	`CalendarEvent`	`String`	Returns the location text from the event. Returns an empty string if none is set.
`event.getMyStatus()`	`CalendarEvent`	`EventStatus`	Returns your RSVP status: ACCEPTED, DECLINED, MAYBE, or INVITED. Throws if you are the event owner — handle with try/catch.

C JavaScript Built-ins & Concepts

Apps Script runs on a JavaScript engine. These standard JavaScript features are used in the script.

SYNTAX / FUNCTION	CATEGORY	RETURNS	WHAT IT DOES
`parseInt(string)`	`Global fn`	`Number`	Converts a string like `'30'` into the integer 30. Returns NaN if it cannot be parsed.
`isNaN(value)`	`Global fn`	`Boolean`	Returns true if the value is NaN (Not a Number). Used to validate that the user typed a real number.

SYNTAX / FUNCTION	CATEGORY	RETURNS	WHAT IT DOES
`new Date()`	Date object	Date	Creates a Date object representing the current moment. Apps Script uses JavaScript's native Date class.
`date.setDate(n)`	Date method	Number	Sets the day-of-month on a Date object. Passing `getDate() + days` advances the date forward.
`date.getDate()`	Date method	Number	Returns the day-of-month (1–31). Combined with setDate() to calculate a future date.
`array.push(item)`	Array method	Number	Appends items to the end of an array. Used here to build the eventList row by row before the batch write.
`(end - start) / 3600000`	Arithmetic	Number	Subtracting two Date objects yields milliseconds. Dividing by 3,600,000 converts to hours.
`number.toFixed(2)`	Number method	String	Rounds to 2 decimal places and returns a string — e.g. 1.5666 becomes `'1.57'`.
`for (var i = 0; ...)`	Control flow	—	A classic for-loop. Iterates from i=0 to events.length−1, processing one event per pass.
`try { } catch(err) { }`	Error handling	—	Wraps code that might throw. If getMyStatus() throws (user is event owner), execution jumps to catch where a fallback value is set.

SYNTAX / FUNCTION	CATEGORY	RETURNS	WHAT IT DOES
`condition ? val1 : val2`	Ternary op.	any	Compact if/else. If condition is true the result is val1, otherwise val2. Used for all-day event duration logic.
`function name(param) { }`	Function decl.	—	Declares a named, reusable block of code. In Apps Script, function names are used as string references in menu wiring.
`var name = value`	Variable decl.	—	Declares a function-scoped variable. Apps Script also supports modern `let` and `const`.

D Key Apps Script Concepts

Foundational concepts behind how this script works within the Google Apps Script environment.

Simple Triggers — onOpen()

`onOpen()` is a reserved function name. Google calls it automatically each time the spreadsheet opens — no setup required. Other triggers include `onEdit()` and `onFormSubmit()`.

Authorisation Scopes

Apps Script requests permission scopes the first time a user runs a function that needs them. This script needs **spreadsheets** and **calendar.readonly**. Google shows these on the OAuth consent screen.

Batch vs. Row-by-Row Writes

Calling `setValues()` once with a 2D array is dramatically faster than calling `appendRow()` in a loop. Each row-by-row call is a separate network round-trip to Google's servers.

Execution Environment & Limits

Apps Script runs server-side on Google's infrastructure, not in your browser. Scripts have a maximum execution time of **6 minutes** per run. For large date ranges, consider splitting into smaller chunks.
